Experiment 9: Job Queue Simulation

Aim:

Queues are frequently used in computer programming, and a typical eg. is the creation of a job queue by an OS. If the OS does not use priorities, then the jobs are processed in the order they enter the system. WACP for simulating a job queue. Write functions to add a job and delete a job.

Objectives:

i. Understand the concept of queue and fifo.
ii. Implement a linear queue using an array.
iii. Implement functions to add and delete jobs.
iv. Simulate processing of jobs in the order they enter the system.
v. Analyze the time and space complexity of queue operations.

Implementation

- Platform

i. 64-bit Open source Linux or its derivatives.
ii. Open source C programming tool like gcc/Eclipse Editor.

- Test Conditions

Test Case	Input	Expected Output
TC01	Add one job	Job added successfully
TC02	Add multiple jobs	Jobs stored in fifo order
TC03	Delete a job	First entered job is deleted/processed
TC04	Delete from empty queue	Queue underflow message
TC05	Add job when queue is full	Queue overflow message
TC06	Display pending jobs	All jobs display from front to rear

- Pseudo Code

```
CONSTANT MAX_SIZE = 5

DEFINE STRUCTURE Job:
        job_id AS integer
        job_name as string

declare queue[MAX_SIZE] as array of Job
declare front as integer = -1
declare rear as integer = -1

function add_job(new_job):
        if rear == MAX_SIZE -1 then
            print "Queue Overflow! Job Queue is full. Cannot add job."
            return
        end if

        if front == -1 then
            front = 0
        end if

        rear = rear +1
        queue[rear] = new_job
        print "Job successfully added to the queue."
end function

function delete_job():
        if front == -1 or front > rear then
            print "Queue Underflow! No pending jobs to process."
            front = -1
```

```
        rear = -1
        return
    end if

    declare processed_job as Job
    processed_job = queue[front]
    print "Processing and removing Job ID:", processed_job.job_id,
    ", Name:", processed_job.job_name

    front = front + 1

    if front > rear then
        front = -1
        rear = -1
    end if
end function

function display_jobs():
    if front == -1 or front > rear then
        print "Job Queue is empty. No pending jobs."
        return
    end if

    print "--- Pending Jobs in Queue ---"
    for i from front to rear: do
        print "Job ID:", queue[i].job_id, "| Name:", queue[i].job_name
    end for
end function
```

```
begin main
    declare choice as integer
    declare temp_job as Job

    repeat
        print "\n -- Job Queue Simulator --"
        print "1. Add Job (Enqueue)"
        print "2. Process / Delete Job (Dequeue)"
        print "3. Display pending jobs"
        print "4. Exit"
        print "Enter your choice:"
        read choice

    switch choice do
        case 1:
            print "Enter Job ID:"
            read temp_job.job_id
            print "Enter Job Name:"
            read temp_job.job_name
            add_job (temp_job)
            break
        case 2:
            delete_job()
            break
        case 3:
            display_jobs()
            break
        case 4:
            print "Exiting program"
            break
```

```
            default:
                print "Invalid choice!"
        end switch
    until choice == 4
end main
```

- Time and Space Complexity:

i. addJob() → O(1): Constant time insertion at the rear.
ii. deleteJob()→O(1): Constant time deletion at the front.
iii. displayJobs()→O(N): Traverses all jobs from front to rear.
iv. Array storage requires space for a maximum of N jobs, where N=MAX.: O(N)

Conclusion:

The job queue simulation demonstrates the fifo principle of a linear queue. Jobs are added at the rear and processed/deleted from the front in the same order in which they enter the system.

FAQs:

1. What is a queue?

Ans. A queue is a linear data structure where elements are added at the rear end and removed from the front end.

2. What is fifo?

Ans. Fifo (first in, first out) is an ordering principle where the first elements inserted into the structure is the first one to be deleted/processed.

3. Why is a queue suitable for an OS job queue?

Ans. A queue is suitable for an OS job queue because it processes incoming system tasks in the exact order they arrive when priorities are not used.

4. What is Queue Overflow and Queue Underflow?

Ans. Queue Overflow occurs when attempting to insert an element into a compl-etely full queue, whereas Queue Underflow occurs when trying to delete an element from an empty queue.

5. What are the applications of Queue?

Ans. Queues are applied in OS job scheduling, asynchronous buffer management, printer spooling, and CPU task management.

6. What is a Circular Queue?

Ans. A Circular Queue is a linear data structure in which the last array index is logically connected back to the first index to form a continuous ring.

7. Why is a Circular Queue preferred over a linear queue?

Ans. A circular queue is preferred over a linear queue because it efficiently reuses vacated positions at the front after deletion, preventing the memory wastage.

8. What principle does a Circular Queue follow?

Ans. A circular queue follows the fifo principle using circular wrap-around indexing.

9. What are the basic operations performed on a Circular Queue?

Ans. The primary operations are Enqueue and Dequeue.

10. What is an Overflow & Underflow condition in a Circular Queue?

Ans. In a Circular Queue, overflow occurs when (rear+1)%MAX == front, while underflow occurs when front == -1

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX 5  // Maximum capacity of the job queue

// Structure definition according to Assignment 9 pseudocode
typedef struct {
    int job_id;
    char job_name[50];
} Job;

Job queue[MAX];
int front = -1;
int rear = -1;

// Function to add a job to the rear of the queue
void addJob(Job new_job) {
    if (rear == MAX - 1) {
        printf("\nQueue Overflow! Job Queue is full. Cannot add job.\n");
        return;
    }
    if (front == -1) {
        front = 0;
    }
    rear++;
    queue[rear] = new_job;
    printf("\nJob successfully added to the queue.\n");
}

// Function to delete/process a job from the front of the queue
void deleteJob() {
    if (front == -1 || front > rear) {
        printf("\nQueue Underflow! No pending jobs to process.\n");
```

```
        return;
    }
    printf("\nProcessing and removing Job ID: %d, Name: %s\n",
queue[front].job_id, queue[front].job_name);
    front++;

    // Reset queue indices when all jobs have been processed
    if (front > rear) {
        front = -1;
        rear = -1;
    }
}

// Function to display all pending jobs in FIFO order
void displayJobs() {
    if (front == -1 || front > rear) {
        printf("\nJob Queue is empty. No pending jobs.\n");
        return;
    }
    printf("\n--- Pending Jobs in Queue ---\n");
    for (int i = front; i <= rear; i++) {
        printf("Job ID: %d | Name: %s\n", queue[i].job_id, queue[i].job_name);
    }
}

int main() {
    int choice;
    Job temp_job;

    while (1) {
        printf("\n========================================\n");
        printf("     OPERATING SYSTEM JOB QUEUE         \n");
        printf("========================================\n");
        printf("1. Add Job (Enqueue)\n");
        printf("2. Process/Delete Job (Dequeue)\n");
        printf("3. Display Pending Jobs\n");
```

```
        printf("4. Exit\n");
        printf("Enter your choice: ");
        if (scanf("%d", &choice) != 1) break;

        switch (choice) {
            case 1:
                printf("Enter Job ID: ");
                scanf("%d", &temp_job.job_id);
                printf("Enter Job Name: ");
                scanf("%49s", temp_job.job_name);
                addJob(temp_job);
                break;

            case 2:
                deleteJob();
                break;

            case 3:
                displayJobs();
                break;

            case 4:
                printf("\nExiting program. Goodbye!\n");
                exit(0);

            default:
                printf("\nInvalid choice! Please select a valid option.\n");
        }
    }
    return 0;
}
```

Input:

2
1
101
PrintTask
1
102
Compiler
1
103
AudioDriver
1
104
VirusScan
1
105
Backup
3
1
106
Download

2

3

4

Output:

```
=======================================
     OPERATING SYSTEM JOB QUEUE
=======================================
1. Add Job (Enqueue)
2. Process/Delete Job (Dequeue)
3. Display Pending Jobs
4. Exit
Enter your choice:2


Queue Underflow! No pending jobs to process.

=======================================
     OPERATING SYSTEM JOB QUEUE
=======================================
1. Add Job (Enqueue)
2. Process/Delete Job (Dequeue)
3. Display Pending Jobs
4. Exit
Enter your choice:1

Enter Job ID:101

Enter Job Name:PrintTask
```

```
Job successfully added to the queue.

=======================================
     OPERATING SYSTEM JOB QUEUE
=======================================
1. Add Job (Enqueue)
2. Process/Delete Job (Dequeue)
3. Display Pending Jobs
4. Exit
Enter your choice:1

Enter Job ID:102

Enter Job Name:Compiler


Job successfully added to the queue.

=======================================
     OPERATING SYSTEM JOB QUEUE
=======================================
1. Add Job (Enqueue)
2. Process/Delete Job (Dequeue)
3. Display Pending Jobs
4. Exit
```

```
Enter your choice:1

Enter Job ID:103

Enter Job Name:AudioDriver
```

```
Job successfully added to the queue.

========================================
     OPERATING SYSTEM JOB QUEUE
========================================
1. Add Job (Enqueue)
2. Process/Delete Job (Dequeue)
3. Display Pending Jobs
4. Exit
Enter your choice:1

Enter Job ID:104

Enter Job Name:VirusScan


Job successfully added to the queue.
```

```
========================================
     OPERATING SYSTEM JOB QUEUE
========================================
1. Add Job (Enqueue)
2. Process/Delete Job (Dequeue)
3. Display Pending Jobs
4. Exit
Enter your choice:1

Enter Job ID:105

Enter Job Name:Backup


Job successfully added to the queue.

========================================
     OPERATING SYSTEM JOB QUEUE
========================================
1. Add Job (Enqueue)
2. Process/Delete Job (Dequeue)
3. Display Pending Jobs
4. Exit
Enter your choice:3
```

```
 --- Pending Jobs in Queue ---
 Job ID: 101 | Name: PrintTask
 Job ID: 102 | Name: Compiler
 Job ID: 103 | Name: AudioDriver
 Job ID: 104 | Name: VirusScan
 Job ID: 105 | Name: Backup

 ========================================
      OPERATING SYSTEM JOB QUEUE
 ========================================
 1. Add Job (Enqueue)
 2. Process/Delete Job (Dequeue)
 3. Display Pending Jobs
 4. Exit
 Enter your choice:1

 Enter Job ID:106

 Enter Job Name:Download


 Queue Overflow! Job Queue is full. Cannot add job.
```

```
========================================
     OPERATING SYSTEM JOB QUEUE
========================================
1. Add Job (Enqueue)
2. Process/Delete Job (Dequeue)
3. Display Pending Jobs
4. Exit
Enter your choice:2


Processing and removing Job ID: 101, Name: PrintTask

========================================
     OPERATING SYSTEM JOB QUEUE
========================================
1. Add Job (Enqueue)
2. Process/Delete Job (Dequeue)
3. Display Pending Jobs
4. Exit
Enter your choice:3
```

```
--- Pending Jobs in Queue ---
Job ID: 102 | Name: Compiler
Job ID: 103 | Name: AudioDriver
Job ID: 104 | Name: VirusScan
Job ID: 105 | Name: Backup

========================================
     OPERATING SYSTEM JOB QUEUE
========================================
1. Add Job (Enqueue)
2. Process/Delete Job (Dequeue)
3. Display Pending Jobs
4. Exit
Enter your choice:4


Exiting program. Goodbye!

Process finished with exit code 0
```